
Bike demand predictions

Project in Statistical Machine Learning

Ida Larsson

Karolina Ilieva

Nicole Mikha

Elias Kassie

Abstract

The aim of this project is to predict how the demand of bikes provided by Capital Bikeshare in Washington, D.C., depends on various time and weather conditions. Five different machine learning models were applied to a training data set containing 1600 data points with 15 different features. The five models were kNN, logistic regression, LDA, decision trees and random forests. Each model was tuned on 75% of the data set and the remaining 25% were used as test data to compare the performance of the different models by calculating the F1-score. The random forest proved to be the best performing out of the tested models.

1 Introduction

In Washington, D.C., bikes are available to rent, but the demand varies. In order to know if the company should increase their stock or not it is desired to predict how the demand depends on different factors such as different weather conditions or time of day etc. The demand will be predicted from a training data set that consists of 1600 data points with 15 different features. By applying different machine learning models we can predict if the demand will be high or low when presented with new unlabeled test data.

2 Data analysis

Among the 15 features, the numerical features are the following: temp, dew, humidity, precip, snow, snow_depth, windspeed, cloudcover and visibility. The categorical features are likewise represented as numbers in the table, however they indicate groups rather than numerical data: hour_of_day, day_of_week, month, holiday, weekday and summertime.

In the training data set the categorical features are represented by numbers. In order to make the different values for these features to be equal, one-hot encoding was applied. Additionally some models need scaling of the data to perform better. If the numerical values between different features differ a lot in size some features may influence the predictions more than others. Then scaling can be done so all features can be considered equally. For this project all features were used, without combining them into new subsets of features.

According to figure 1a, the demand is typically highest during the afternoon, around 5 pm to 6 pm, with the demand being high 70% of the time in data points observed around these hours. Figure 1b illustrates that the bike demand is significantly higher on Saturdays, about 30% of the time, and on Sundays, just over 20% of the time, whereas all weekdays show under 20% high demand. It is clear from figure 1c that wintertime typically entails low bike demand, with high demand being very close to 0% of the observed data points, and that it peaks in April, June and September, at about 30% of the observed data points. In figure 1e, there is no distinguishable trend. Both holidays and regular days show a relative demand of about 20%.

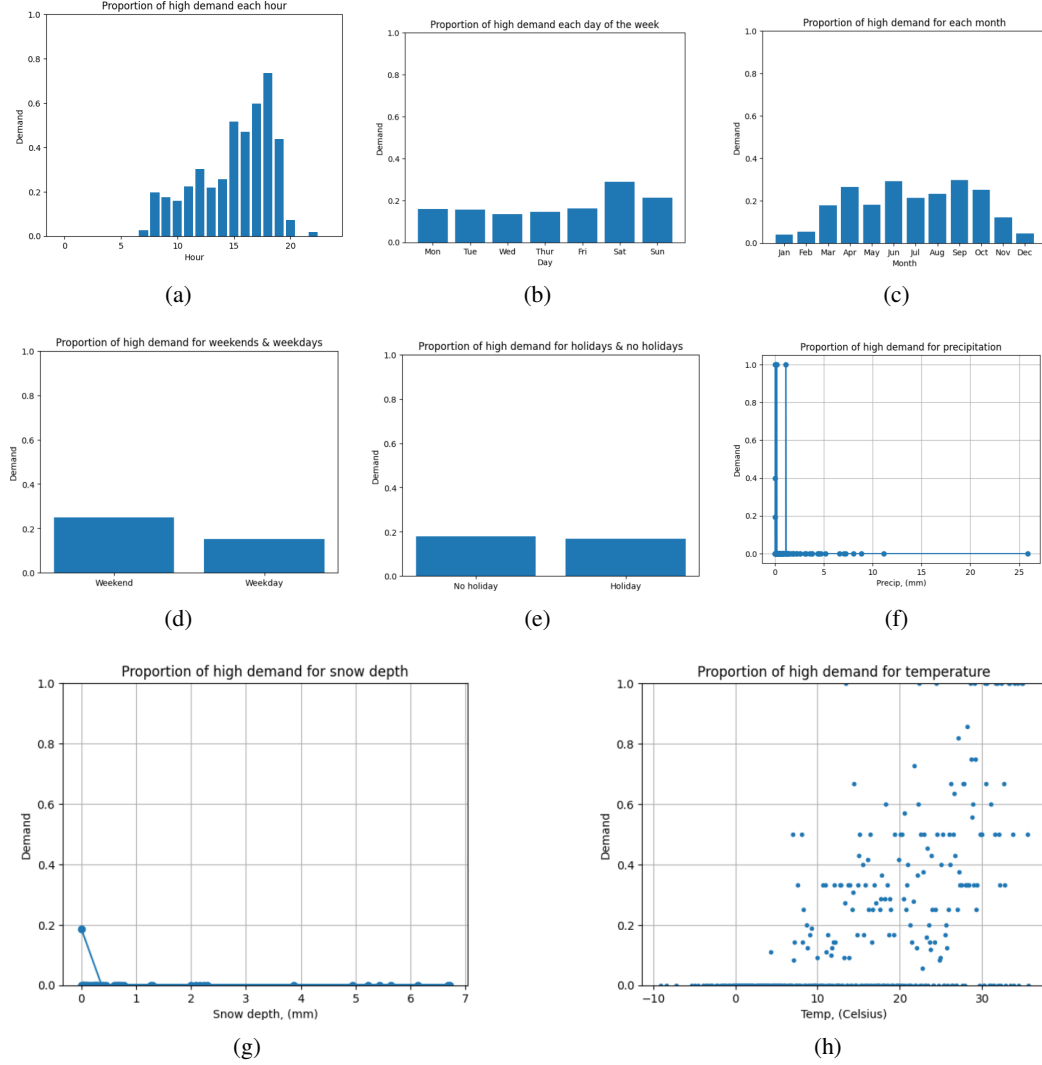


Figure 1: Demand depending on a selected number of features

3 Model development

3.1 Cross-validation

In cross-validation the data is divided into k folds and one fold is kept as validation data while the rest is used to train the model. A prediction is then made and a misclassification error can be calculated. The process is then repeated k times and every fold is used as the validation data exactly once, with the rest of the folds as training data. The benefit of using cross-validation instead of a single validation split, is that it gives a more reliable estimate of generalisation performance, since every observation is used for validation exactly once. The lowest misclassification error was used to determine the best value for the hyperparameters for each model.

3.2 KNN

The k -nearest neighbor model can be used for both regression and classification problems. It is a non-parametric and distance based model. The training of the model includes choosing a value for the hyperparameter k , which stands for the number of neighbors that will be included for the prediction of the new value $\hat{y}(x_*)$. Another choice that has to be made is how to calculate the distance between

data points in order to be able to determine which points that are the k nearest. A common choice is to use the Euclidean distance, shown in equation 1, which was implemented in this problem.

$$d(x_i, x_*) = \|x_i - x_*\|_2 = \sqrt{(x_i - x_*)(x_i - x_*)} \quad (1)$$

The chosen k can impact how well the model performs. Small values for k risks overfitting. If the chosen k is small the model can adapt too well to the training data. This would result in good predictions if the model is applied to the training data, but not when it is applied to the unseen test data. A too large k can also make the model perform less well and result in a biased model. In that case some classes may be predicted incorrectly because they are the majority of the k neighbors, even though there is another class that is closer to the test input and therefore a more accurate prediction.

To do the prediction the distance between the test input x_* and the training inputs and a set of the k nearest data points will be considered for the actual prediction. How the prediction is made depends on if the problem is a classification or regression problem. In the regression case it is common to do the prediction based on a calculated average of the k nearest neighbors, (equation 2). In this case the problem is a binary classification problem. For classification problems the most common approach for the prediction is to use the majority vote, (equation 2).

$$\hat{y}(x_*) = \begin{cases} \text{Average}\{y_j : j \in N_*\} & \text{Regression problem} \\ \text{MajorityVote}\{y_j : j \in N_*\} & \text{Classification problems} \end{cases} \quad (2)$$

When implementing the kNN on this problem cross-validation was used in order to choose an appropriate value of k . Since kNN is a distance based model it is important to do scaling before the training of the model. The scaled training data was split into 10 folds and each was used as validation data once with the rest as training data. Then kNN was applied and tested for k values between 1 and 200. For each run the proportion of wrong predictions were calculated. The average of this for each value of k was calculated, to find which k resulted in the smallest misclassification error. This was seen to be $k = 9$. The new kNN model with $k = 9$ was then trained on the whole X_{train} , (no cross-validation), resulting in an accuracy of 85%, as shown in 1 under shown in 4.1 Evaluation criteria.

3.3 Logistic regression

Logistic regression for binary classification is a classification model that predicts the class membership, $\{-1, 1\}$, given a set of parameters. The method uses a linear prediction, but the output is non-linear. After numerically determining the parameter vector $\hat{\theta}$ (see equation 3 below), the output is converted into a probability, a value between 0 and 1, using the sigmoid (logistic) function (see equation 4 below). Class prediction is determined by thresholding (see equation 5 below). Usually $r = 0.5$ is used to minimize all missclassifications, but it might be tuned in cases where one missclassifications is 'less acceptable' than the other, for example, in models used for medical applications.

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ln \left(1 + e^{-y_i \theta^T x_i} \right) \quad (3)$$

$$g(x) = \frac{e^{\theta^T x}}{1 + e^{\theta^T x}} \quad (4)$$

$$\hat{y}(x_*) = \begin{cases} 1 & \text{if } g(x) > r \\ -1 & \text{if } g(x) \leq r \end{cases} \quad (5)$$

The model was trained using the designated processed training data and tested using the test data. 'LogisticRegression' from the sklearn-package was used to apply the method to the data set. The missclassification error was minimized by hyperparameter tuning through 10-fold cross validation was performed to decide the regularization strength and the penalty type. The optimal parameters were then used to train the final model, ensuring a balance between bias and variance.

3.4 Linear discriminative analysis

Linear discriminant analysis, LDA, is a generative classifier that is parametric. A generative model describes data based on the joint distribution of inputs and outputs, $P(y, x)$. The method for LDA is based upon the conditional probability $P(y|x)$ shown in equation 6 below.

$$P(y|x) = \frac{P(y, x)}{P(x)} = \frac{P(y, x)}{\sum_{j=1}^M P(x, y = j)} \quad (6)$$

Further the inputs x of each class m is assumed to be Gaussian distributed with mean μ_m and the covariance matrix Σ_m . In LDA the covariance matrices for each class is assumed to be the same, meaning that the spread and shape of each class is assumed to be the same. The right-hand equation of 6 can be rewritten as equation 7.

$$p(y = m | x_*) = \frac{\hat{\pi}_m \mathcal{N}(x_* | \hat{\mu}_m, \hat{\Sigma}_m)}{\sum_{j=1}^M \hat{\pi}_j \mathcal{N}(x_* | \hat{\mu}_j, \hat{\Sigma}_j)} \quad (7)$$

If $\hat{y}$ is predicted on the basis of x_* , the highest probability $P(\hat{y}|x_*)$ is sought. By simplifying the expression and replacing the Gaussian distribution with its probability density function the expression in 8 is obtained. The expression in the brackets is called the discriminant function.

$$\hat{y}_* = \left\{ \log \hat{\pi}_m - \frac{1}{2} x^\top \hat{\Sigma}^{-1} x + x^\top \hat{\Sigma}^{-1} \hat{\mu}_m - \frac{1}{2} \hat{\mu}_m^\top \hat{\Sigma}^{-1} \hat{\mu}_m \right\} \quad (8)$$

When implementing the LDA model with scikit learn three options for solvers are given. These options are singular value decomposition (SVD), least squares solution (lsqr) and eigenvalue decomposition (eigen). For lsqr a least square solution is used together with the covariance matrix implicitly to find a vector ω , a vector that defines which linear combinations of features best separate the classes. The eigen solver explicitly uses the covariance matrix in an eigenvalue problem to solve for ω . This however implies that the covariance matrix is invertible, which it is not if the features have a higher dimension than the output. In both eigen and lsqr shrinkage is allowed, this stabilises the covariance matrix and the computations of ω . The svd solver finds ω by projecting the data onto the subspace with the largest variance via SVD. The covariance matrix is not explicitly computed, so shrinkage cannot be used.

With cross-validation using ten folds on the processed training dataset and evaluation based on accuracy the best solver was picked. This showed that lsqr with auto shrinkage was the best solver since the mean misclassification of the cross-validations was the lowest. The lsqr solver for LDA was then trained on the entire processed training dataset and tested on the processed test dataset. This gave an accuracy of 87.3 % as shown in 1 in 4.1 Evaluation criteria. The performance was further evaluated and compared to the other methods in a boxplot (see appendix 4) and using the F1-score. The F1-score is also shown in 1. The decision rule (r-value) of 0.5 was used throughout the entire testing and training as it was set to default. Since the data were processed with one-hot encoding and a default threshold (r-value) was set, LDA could be applied to categorical features.

3.5 Tree-based methods

Two tree-based models were developed for this problem: a single Decision Tree and a Random Forest. The decision tree serves as a transparent baseline, while the random forest ensemble is the primary model aimed at maximising predictive accuracy.

3.5.1 Decision trees

As a first step, a fully grown decision tree (without restricting the depth) was trained on the training data. This provides a baseline and allows us to observe potential overfitting.

A decision tree makes predictions by performing recursive binary splitting of the feature space. At each internal node, the model identifies a single feature X_j and a threshold s that splits the data into two regions $\{X|x_j < s\}$ and $\{X|x_j \geq s\}$, such that the Gini Impurity is minimized. The Gini

index, defined as equation 9, measures the "purity" of a node; a lower value indicates that a node is predominantly composed of a single class.

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}) \quad (9)$$

For a new observation, the model follows the path of logic gates (e.g., "Is the hour > 7.5?") until it reaches a terminal "leaf" node. The final classification is determined by a majority vote of the training observations assigned to that leaf. The first four levels of the tuned tree can be seen in figure 2 in the appendix.

In our model, the training error was very low, (0.0000), while the test error was noticeably higher, (0.1300), indicating that the unrestricted tree was overfitting. This motivated tuning the tree complexity.

10-fold cross-validation was repeated for tree depths ranging from 1 to 30. The average validation error across the 10 folds was computed for each depth. The depth with the lowest average cross-validation error was selected as the optimal tree complexity.

After selecting the optimal depth, a new decision tree was trained on the entire training dataset using this depth. The model was then evaluated on the independent test dataset. Performance was assessed using the test error (missclassification rate), accuracy, confusion matrix and the F1-score for the high demand class.

3.5.2 Random tree

While a single decision tree is easy to interpret, it can be unstable: small changes in the data may produce very different trees. Random Forest addresses this issue by combining many decision trees into an ensemble. Each individual tree makes a prediction, and the final classification is determined by majority voting across all trees. This approach reduces variance and typically improves predictive performance compared to a single tree.

The number of trees (n_estimators) controls how many models are included in the ensemble. Increasing the number of trees generally reduces variance, but after a certain point, the improvement becomes negligible.

To study this, the test error was plotted as a function of the number of trees. The error stabilised around approximately 100 trees, indicating diminishing returns beyond this value, (see fig. 3a in appendix). Therefore, 100 trees were selected for the final model.

Another important hyperparameter is max_features, which determines how many features are randomly considered at each split. If too many features are considered, the trees become more similar to each other, reducing the benefit of randomness. If too few features are used, the trees may become too weak.

To select an appropriate value, 10-fold cross-validation was applied again. For each candidate value of max_features, the average validation error across folds was computed. The value with the lowest cross-validation error was chosen for the final model, (see fig. 3b in appendix).

Using the selected hyperparameters (number of trees and max_features), the final Random Forest model was trained on the entire training dataset and evaluated on the test dataset. The evaluation included:

- Training and test error
- Accuracy
- Confusion matrix
- F1-score for high bike demand
- Feature importance

The F1-score is especially relevant in this application since detecting high-demand periods is more critical than predicting low-demand periods. Random Forest typically achieved a higher F1-score than the single decision tree, indicating better balance between precision and recall.

Feature scaling was not applied to the tree-based models as these algorithms are naturally scale-invariant. Unlike distance-based methods, decision trees and random forests utilize threshold-based splits to partition the feature space, meaning the absolute magnitude of numerical variables does not influence the model’s structure or predictive accuracy.

3.6 Naive model

An additional naive model was also implemented. This model simply predicts high or low demand randomly for each point in the data set. The reason to include such a model is to compare it to the performance of the rest, so it can be ruled out that their good performance would be because of randomness.

4 Model selection

To determine the most suitable model for predicting high bike demand, the performance of all implemented methods was systematically compared. The evaluated models were:

- k-Nearest Neighbours (kNN)
- Logistic Regression
- Linear Discriminant Analysis (LDA)
- Decision Tree
- Random Forest
- Naive model

Since the primary objective of this project is to identify periods of high bike demand correctly, model comparison was based not only on overall accuracy but also on the F1-score for the high-demand class.

4.1 Evaluation criteria

Although accuracy provides a general measure of predictive performance, it can be misleading in classification problems with class-imbalanced data. If high-demand days occur less frequently than low-demand days, a model could achieve high accuracy simply by predicting the majority class.

For this reason, the F1-score was chosen as the primary evaluation metric. The F1-score combines precision and recall and therefore measures how well a model balances:

- Correctly detecting high-demand days (recall)
- Avoiding false alarms (precision)

A model with a high F1-score is therefore effective at both identifying true high-demand periods and limiting incorrect high-demand predictions.

Table 1: Comparison of classification models

Model	Test Accuracy	F1-score (High Demand)
kNN	85%	0.44
Logistic regression	87.5%	0.56
LDA	87.3%	0.60
Decision Tree	88.5%	0.68
Random Forest	91%	0.72
Naive	48%	0.27

Among all methods, the Random Forest model achieved the highest F1-score, indicating superior ability to detect high-demand periods while maintaining precision.

A boxplot comparison of cross-validation scores further illustrated that Random Forest not only achieved high average performance but also demonstrated lower variance compared to individual trees, (see fig. 4 in appendix).

4.2 Final model choice

Based on the evaluation the Random Forest had the highest F1-score, (see table 1), and it maintained a strong overall accuracy. It also reduced overfitting compared to a single decision tree. Therefore, Random Forest was selected as the final model for predicting high bike demand.

5 Conclusion

The objective of this project was to predict whether bike demand in Washington, D.C. would increase based on weather conditions, time-related features, and seasonal patterns. Exploratory data analysis revealed some clear trends. The demand peaks in late afternoon hours and weekends show higher demand than weekdays. Winter months exhibit very low demand and rain or snow significantly reduces demand.

Different classification methods were implemented and compared using a consistent cross-validation framework. Special attention was given to the high-demand class, since correctly predicting these periods is practically important for inventory decisions. The results indicate that:

- Linear models (Logistic Regression and LDA) provide stable baseline performance
- kNN performs well but is sensitive to scaling and the choice of k
- A single decision tree is interpretable but prone to overfitting
- Random Forest achieves the best predictive performance and strongest F1-score

The ensemble nature of Random Forest reduces variance and improves generalisation, making it the most suitable model for this problem.

5.1 Practical implications

Using the Random Forest model, the company can more reliably identify periods of high bike demand and adjust inventory accordingly. This can improve the bike availability during peak times and reduce missed rental opportunities as well as optimise resource allocation.

5.2 Limitations and future work

While Random Forest performed best among the tested models, further improvements could include:

- Performing more advanced hyperparameter optimisation
- Incorporating additional temporal features
- Evaluating cost-sensitive classification if misclassification costs differ

References

[1] Lindholm, A., Wahlström, N., Lindsten, F. & Schön, T.B. (2022) *Machine Learning - A First Course for Engineers and Scientists*. Cambridge University Press. <https://smlbook.org/book/sml-book-draft-latest.pdf>

A Appendix

A.1 Figures

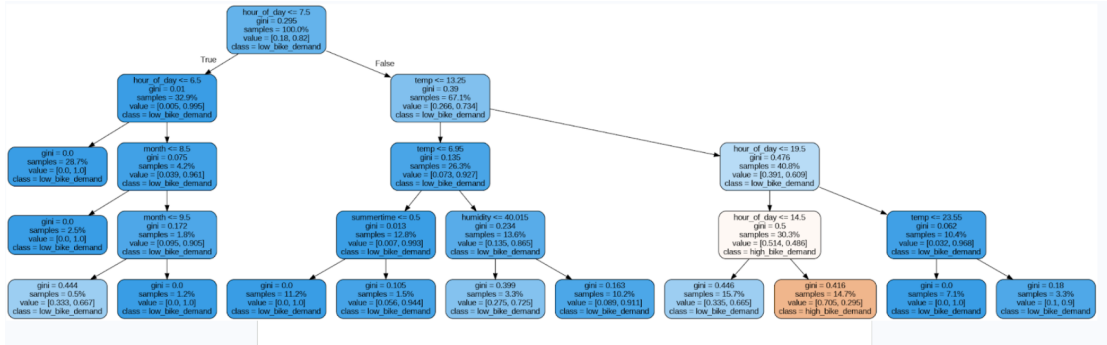
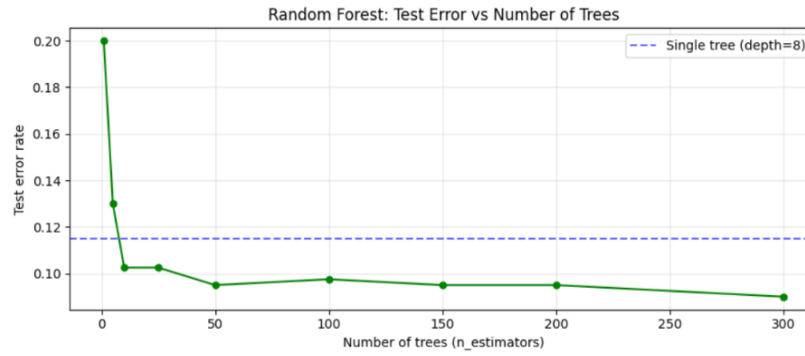
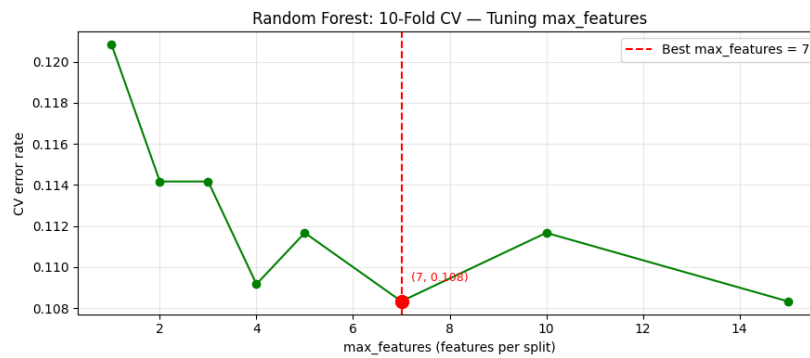


Figure 2: The first four levels of the tuned tree, highlighting how the features `hour_of_day` and `temp` acts as the primary root-level splitters.



(a)



(b)

Figure 3: Evaluation of random forest

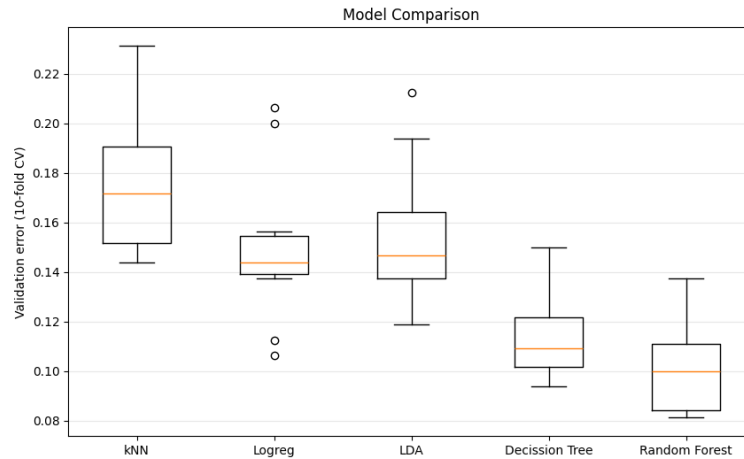


Figure 4: Comparison between models

A.2 Code

A.2.1 Import of data and data analysis

```

1 # -*- coding: utf-8 -*-
2 """SML-project.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/16
8     Wt_Y5DLBPdXNRI80mvMuVf390UWa56
9 """
10 # Installing graphviz
11 !apt-get install graphviz -q
12 !pip install graphviz -q
13
14 ## Import necessary packages
15 import pandas as pd
16 import matplotlib.pyplot as plt
17 import matplotlib.patches as mpatches
18 import numpy as np
19
20 import sklearn.neighbors as skl_nb
21 import sklearn.linear_model as skl_lm
22 import sklearn.discriminant_analysis as skl_da
23 import sklearn.model_selection as skl_ms
24
25 # Tree Based Methods
26 from sklearn import tree
27 from sklearn.tree import DecisionTreeClassifier
28 from sklearn.ensemble import RandomForestClassifier
29 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
30 from sklearn.metrics import f1_score, classification_report
31 import graphviz
32
33 ## import the training data
34 url = 'https://drive.google.com/uc?export=download&id=1
35     Aj8Nzvwg3QJNFZZBuZz4M69Rzv8Lti6y'
36 data = pd.read_csv(url, na_values='?').dropna()
37 # data.info()
38 # print(data)

```

```

39 ## -----Plot for demand depending on hours of the day-----
40
41 hour_count = {}
42
43 for h in range(0, 24):
44     count_high = data[(data['hour_of_day'] == h) & (data['increase_stock']
45     ''] == 'high_bike_demand')].shape[0]
46     count_tot = data[(data['hour_of_day'] == h)].shape[0]
47     hour_count[h] = count_high/count_tot
48
49 plt.bar(hour_count.keys(), hour_count.values())
50 plt.xlabel('Hour')
51 plt.ylabel('Demand')
52 plt.title('Proportion of high demand each hour')
53 plt.ylim(0,1)
54 plt.show()
55
56 ## -----Plot for demand depending on days of the week-----
57
58 day_count = {}
59
60 for d in range(0, 7):
61     count_high = data[(data['day_of_week'] == d) & (data['increase_stock']
62     ''] == 'high_bike_demand')].shape[0]
63     count_tot = data[(data['day_of_week'] == d)].shape[0]
64     day_count[d] = count_high/count_tot
65
66 plt.bar(day_count.keys(), day_count.values())
67 plt.xlabel('Day')
68 plt.xticks([0, 1, 2, 3, 4, 5, 6], ['Mon', 'Tue', 'Wed', 'Thur', 'Fri',
69     'Sat', 'Sun'])
70 plt.ylabel('Demand')
71 plt.title('Proportion of high demand each day of the week')
72 plt.ylim(0, 1)
73 plt.show()
74
75 ## -----Plot for demand depending on the month-----
76
77 month_count = {}
78
79 for m in range(1, 13):
80     count_high = data[(data['month'] == m) & (data['increase_stock'] ==
81     'high_bike_demand')].shape[0]
82     count_tot = data[(data['month'] == m)].shape[0]
83     month_count[m] = count_high/count_tot
84
85 plt.bar(month_count.keys(), month_count.values())
86 plt.xticks(range(1, 13), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', '
87     Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
88 plt.xlabel('Month')
89 plt.ylabel('Demand')
90 plt.title('Proportion of high demand for each month')
91 plt.ylim(0,1)
92 plt.show()
93
94 ## -----Plot for demand depending on weekend/weekday-----
95
96 weekday_count = {}
97
98 for w in range(0, 2):
99     count_high = data[(data['weekday'] == w) & (data['increase_stock']
100     ''] == 'high_bike_demand')].shape[0]

```

```

98     count_tot = data[(data['weekday'] == w)].shape[0]
99     weekday_count[w] = count_high/count_tot
100
101
102 plt.bar(weekday_count.keys(), weekday_count.values())
103 plt.title('Proportion of high demand for weekends & weekdays')
104 plt.ylabel('Demand')
105 plt.xticks([0, 1], ['Weekend', 'Weekday'])
106 plt.ylim(0,1)
107 plt.show()
108
109 ## -----Plot for demand depending on holiday/no holiday-----
110
111 holiday_count = {}
112
113 for h in range(0, 2):
114     count_high = data[(data['holiday'] == h) & (data['increase_stock']
115 == 'high_bike_demand')].shape[0]
116     count_tot = data[(data['holiday'] == h)].shape[0]
117     holiday_count[h] = count_high/count_tot
118
119
120 plt.bar(holiday_count.keys(), holiday_count.values())
121 plt.title('Proportion of high demand for holidays & no holidays')
122 plt.ylabel('Demand')
123 plt.xticks([0, 1], ['No holiday', 'Holiday'])
124 plt.ylim(0,1)
125 plt.show()
126
127 """..."""
128
129 ## -----Plot for demand depending on precipitation-----
130
131 precip_count = {}
132
133 for p in data['precip']:
134     count_high = data[(data['precip'] == p) & (data['increase_stock'] ==
135 'high_bike_demand')].shape[0]
136     count_tot = data[(data['precip'] == p)].shape[0]
137
138     precip_count[p] = count_high/count_tot
139
140 precip_count = dict(sorted(precip_count.items()))
141
142 plt.plot(precip_count.keys(), precip_count.values(), '-o')
143 plt.title('Proportion of high demand for precipitation')
144 plt.xlabel('Precip, (mm)')
145 plt.ylabel('Demand')
146 plt.grid()
147 plt.show()
148
149 ## -----Plot for demand depending on snow depth-----
150
151 snow_depth_count = {}
152
153 for s in data['snowdepth']:
154     count_high = data[(data['snowdepth'] == s) & (data['increase_stock']
155 == 'high_bike_demand')].shape[0]
156     count_tot = data[(data['snowdepth'] == s)].shape[0]
157     snow_depth_count[s] = count_high/count_tot
158
159 precip_count = dict(sorted(precip_count.items()))
160
161 plt.plot(snow_depth_count.keys(), snow_depth_count.values(), '-o')

```

```

160 plt.title('Proportion of high demand for snow depth')
161 plt.xlabel('Snow depth, (mm)')
162 plt.ylabel('Demand')
163 plt.ylim(0,1)
164 plt.grid()
165 plt.show()
166
167 ## -----Plot for demand depending on snow depth-----
168
169 temp_count = {}
170
171 for t in data['temp']:
172     count_high = data[(data['temp'] == t) & (data['increase_stock'] == '
173         high_bike_demand')].shape[0]
174     count_tot = data[(data['temp'] == t)].shape[0]
175     temp_count[t] = count_high/count_tot
176
177 precip_count = dict(sorted(precip_count.items()))
178
179 # plt.bar(temp_count.keys(), temp_count.values(), color = 'black')
180 plt.plot(temp_count.keys(), temp_count.values(), '.')
181 plt.title('Proportion of high demand for temperature')
182 plt.xlabel('Temp, (Celsius)')
183 plt.ylabel('Demand')
184 plt.ylim(0,1)
185 plt.grid()
186 plt.show()
187
188 """# split into training and test"""

```

A.2.2 Preprocessing of data

```

1 ## -----Split data into training and test data-----
2 np.random.seed(1)
3
4 trainI = np.random.choice(data.shape[0], size = 1200, replace=False)
5
6 # print(trainI[0:5])
7
8 train = data.iloc[trainI]
9 test = data.drop(trainI)
10
11 # print(test[0:5])
12
13 x_train = train[['hour_of_day', 'day_of_week', 'month',
14                 'holiday', 'weekday', 'summertime', 'temp', 'dew',
15                 'humidity', 'precip', 'snow', 'snowdepth', 'windspeed',
16                 ', 'cloudcover', 'visibility' ]]
17
18 x_test = test[['hour_of_day', 'day_of_week', 'month',
19               'holiday', 'weekday', 'summertime', 'temp', 'dew',
20               'humidity', 'precip', 'snow', 'snowdepth', 'windspeed',
21               ', 'cloudcover', 'visibility' ]]
22
23 y_train = train['increase_stock']
24 y_test = test['increase_stock']
25
26 print(np.shape(x_train))
27
28 # print(x_train[0:5])
29
30 # ----- Normalization and scaling of num & categ. data -----

```

```

31 from sklearn.preprocessing import StandardScaler, OneHotEncoder
32 from sklearn.compose import ColumnTransformer
33
34 n_transf = StandardScaler()
35 c_transf = OneHotEncoder(handle_unknown = 'ignore', sparse_output=
    False)
36
37 preprocessor = ColumnTransformer([
38     ('numerical', n_transf, ['temp', 'dew', 'humidity', 'precip', '
    snow', 'snowdepth', 'windspeed', 'cloudcover', 'visibility']),
39     ('categorical', c_transf, ['hour_of_day', 'day_of_week', 'month',
    'holiday', 'weekday', 'summertime'])
40 ])
41
42 x_train_processed = preprocessor.fit_transform(x_train)
43 x_test_processed = preprocessor.transform(x_test)

```

A.2.3 Implementation of naive model

```

1 ## -----Naive model for comparision-----
2
3 ## randomly selects 'high' or 'low' bike demand for each point in
    x_test
4 pred_naive = np.random.choice(['high_bike_demand', 'low_bike_demand'],
    size = len(x_test))
5
6 ## calculate f1 score and accuracy
7 f1_naive = f1_score(y_test, pred_naive, pos_label='high_bike_demand')
8 missclassification = np.mean(pred_naive != y_test)
9
10
11 print(f'Accuracy: {1-missclassification}')
12 print(f'f1: {f1_naive:.3f}')

```

A.2.4 Implementation of kNN

```

1 ## -----Implementation of kNN-----
2 ## do 10 folds
3 n_fold = 10
4
5 ## array for storing missclassifications for 200 values of K
6 K = np.arange(1, 201)
7 missclassification = np.zeros(len(K))
8
9
10 ## create 10 folds for cross validation
11 cv = skl_ms.KFold(n_splits = n_fold, random_state=1, shuffle=True)
12
13 ## split the scaled data into training and validation data:
14 for train_index, val_index in cv.split(x_train_processed):
15     X_tr, X_val = x_train_processed[train_index], x_train_processed[
        val_index]
16     y_tr, y_val = y_train.iloc[train_index], y_train.iloc[val_index]
17
18
19 ## for each value of k, train the model and do prediction using the
    validation data
20 for j, k in enumerate(K):
21     ## j = index
22     ## k = actual k-value
23
24     model_kNN = skl_nb.KNeighborsClassifier(n_neighbors=k)
25     model_kNN.fit(X_tr, y_tr) ## train on the training folds
26     pred_kNN = model_kNN.predict(X_val) ## use val. data to do
    prediction

```

```

27     missclassification[j] += np.mean(pred_kNN != y_val)    ## calculate
    number of missclassifications
28
29 ## calculate average missclassification for each k
30 missclassification /= n_fold
31
32 missclassification = missclassification.tolist()
33
34 # min_k = missclassification.index(min(missclassification))+1
35 min_err = min(missclassification)
36 min_idx = missclassification.index(min(missclassification))
37
38
39 print(f'Minimum missclassification rate: {min_err:.3f}\nAt position: {
    min_idx}, \
40 (corresponding to k = {min_idx+1})')
41
42 ## plot the missclassification for each k value
43 plt.plot(K, missclassification, label = 'missclassification for each k
    ')
44 plt.title('Cross validation for kNN')
45 plt.xlabel('k value')
46 plt.ylabel('missclassification rate (E_k-fold)')
47 plt.text(K[min_idx], missclassification[min_idx], f'(k, err) = ({K[
    min_idx]}, {missclassification[min_idx]:.3f})')
48 plt.axvline(x = K[min_idx], color = 'grey', label = 'k value with
    lowest missclassification rate')
49 plt.grid()
50 plt.show()
51
52 ## train the model with chosen k on the entire scaled training data
53 k = min_idx+1
54 model_kNN = skl_nb.KNeighborsClassifier(n_neighbors = k)
55 model_kNN.fit(x_train_processed, y_train)
56 prediction_kNN = model_kNN.predict(x_test_processed)
57
58 ## calculate the missclassification rate and f1 score
59 missclassification = np.mean(prediction_kNN != y_test)
60 f1_kNN = f1_score(y_test, prediction_kNN, pos_label='high_bike_demand'
    )
61
62
63 print(f'Performance of kNN with k = {k}: \n\nMissclassification rate:
    {missclassification:.3f} \nAccuracy: {1-missclassification:.3f}')
64 print(f'f1: {f1_kNN:.3f}')

```

A.2.5 Implementation of Logistic regression

```

1 """# **Logistic regression**"""
2
3 from sklearn.linear_model import LogisticRegression
4
5 # 1. Define parameters for tuning
6 # Testing different C (regularisation strength) and different types of
7 # penalties (l1 and l2). Using solver='liblinear', supports both l1
    and l2
8 C_options = np.logspace(-4, 4, 10)
9 penalties = ['l1', 'l2']
10 param_grid = []
11 for p in penalties:
12     for c in C_options:
13         param_grid.append({'penalty': p, 'C': c})
14
15 # 2. 10-fold cross validation
16 n_fold = 10

```

```

17 cv = skl_ms.KFold(n_splits=n_fold, shuffle=True, random_state=1)
18 cv_errors = np.zeros(len(param_grid))
19
20 # 3. Perform tuning
21 for train_index, val_index in cv.split(x_train_processed):
22     # Training and validation folds
23     X_tr, X_val = x_train_processed[train_index], x_train_processed[
24         val_index]
25     y_tr, y_val = y_train.iloc[train_index], y_train.iloc[val_index]
26
27     for i, params in enumerate(param_grid):
28         # Train model for each parameter combinatooon
29         model = LogisticRegression(solver='liblinear',
30                                   penalty=params['penalty'],
31                                   C=params['C'],
32                                   max_iter=1400)
33         model.fit(X_tr, y_tr)
34         # misclassification rate
35         pred = model.predict(X_val)
36         cv_errors[i] += np.mean(pred != y_val)
37
38 # Mean error
39 cv_errors /= n_fold
40
41 # 4. Find the best parameters
42 best_idx = np.argmin(cv_errors)
43 best_params = param_grid[best_idx]
44 print(f"Best parameters: {best_params}")
45 print(f"Smallest cross validation error: {cv_errors[best_idx]:.4f}")
46
47 # 5. Train final model with the best parameters
48 log_model_tuned = LogisticRegression(solver='liblinear',
49                                     penalty=best_params['penalty'],
50                                     C=best_params['C'],
51                                     max_iter=1400)
52 log_model_tuned.fit(x_train_processed, y_train)
53
54 # 6. Validation on test set
55 y_pred = log_model_tuned.predict(x_test_processed)
56 accuracy = (y_pred == y_test).mean()
57 f1 = f1_score(y_test, y_pred, pos_label='high_bike_demand')
58
59 print(f"\nResult on test set:")
60 print(f"Accuracy: {accuracy:.4f}")
61 print(f"F1 Score: {f1:.4f}")

```

A.2.6 Implementation of LDA

```

1 ## -----Implementation of LDA-----
2 # 10-fold CV
3 n_fold = 10
4 cv = skl_ms.KFold(n_splits=n_fold, shuffle=True, random_state=1)
5
6 # List of parameter combinations for diff solvers
7 # Removed the 'eigen' solver options as they caused the LinAlgError
8 # The 'svd' and 'lsqr' solvers are generally more robust.
9 param_list = [
10     {'solver': 'svd', 'shrinkage': None},
11     {'solver': 'lsqr', 'shrinkage': None},
12     {'solver': 'lsqr', 'shrinkage': 'auto'}
13 ]
14
15 # For storage of average missclassification
16 missclassification = np.zeros(len(param_list))
17

```

```

18 #crossvalidation
19
20 for train_index, val_index in cv.split(x_train_processed):
21     X_tr, X_val = x_train_processed[train_index], x_train_processed[
22         val_index]
23     y_tr, y_val = y_train.iloc[train_index], y_train.iloc[val_index]
24
25     for j, params in enumerate(param_list):
26         model = skl_da.LinearDiscriminantAnalysis(
27             solver=params['solver'], shrinkage=params['shrinkage']) #
28         puts in different solvers and shrincages
29         model.fit(X_tr, y_tr)
30         y_pred_LDA = model.predict(X_val)
31         missclassification[j] += np.mean(y_pred_LDA != y_val) #sums
32         missclassification
33
34 # Mean for the folds
35 missclassification /= n_fold
36
37 # Find best parameter
38 min_err = min(missclassification) #smallest mean
39         missclassification
40 min_idx = missclassification.tolist().index(min_err) # index of
41         first occurance of smallest mean classification
42 best_params = param_list[min_idx] #The index corresponds to the order
43         of parameters
44
45 print(f"Minimum error: {min_err:.3f}")
46 print(f"Best parameters: {best_params}")
47
48 """Continue with LDA with chosen solver: 'solver': 'lsqr', 'shrinkage
49     ': 'auto'
50
51 """
52
53 #train model on entire training data
54 model = skl_da.LinearDiscriminantAnalysis(
55     solver='lsqr', shrinkage='auto')
56
57 model.fit(x_train_processed, y_train)
58
59 #predict on entire testdata
60 y_pred_LDA = model.predict(x_test_processed)
61 missclassification_LDA = np.mean(y_pred_LDA != y_test)
62 f1_LDA = f1_score(y_test, y_pred_LDA, pos_label='high_bike_demand')
63
64 print( f'the missclasificication of the LDA is {missclassification_LDA}
65     } and the accuracy is {1- missclassification_LDA}')
66 print(f'f1 score: {f1_LDA}')
```

A.2.7 Implementation of tree based methods

```

1 ## -----Implementation of Tree-based methods-----
2 # first we start by creating the Baseline (Naive) Model for refference
3
4 tree_full = DecisionTreeClassifier(random_state=1) # no depth limit
5 tree_full.fit(x_train, y_train)
6
7 train_err = np.mean(tree_full.predict(x_train) != y_train)
8 test_err = np.mean(tree_full.predict(x_test) != y_test)
9 print(f'Train error: {train_err:.4f}, Test error: {test_err:.4f}')
10
```

```

11 # Candidate depths to evaluate
12
13 depths = np.arange(1, 31) # depths 1 to 30
14 n_fold = 10
15
16 cv = skl.ms.KFold(n_splits=n_fold, random_state=1, shuffle=True)
17 cv_errors = np.zeros(len(depths))
18
19 for train_idx, val_idx in cv.split(x_train):
20     X_tr, X_val = x_train.iloc[train_idx], x_train.iloc[val_idx]
21     y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]
22
23     for j, d in enumerate(depths):
24         m = DecisionTreeClassifier(max_depth=d, random_state=1)
25         m.fit(X_tr, y_tr)
26         cv_errors[j] += np.mean(m.predict(X_val) != y_val)
27
28 cv_errors /= n_fold # average over folds
29
30 best_depth = depths[np.argmin(cv_errors)]
31 best_cv_err = cv_errors[np.argmin(cv_errors)]
32
33 # Plot CV curve
34 plt.figure(figsize=(10, 4))
35 plt.plot(depths, cv_errors, 'b-o', markersize=4, label='CV error')
36 plt.axvline(best_depth, color='red', linestyle='--', label=f'Best
    depth = {best_depth}')
37 plt.scatter(best_depth, best_cv_err, color='red', s=100, zorder=5)
38 plt.text(best_depth + 0.5, best_cv_err + 0.002,
39         f'(depth={best_depth}, err={best_cv_err:.3f})', color='red',
40         fontsize=9)
41 plt.xlabel('max_depth')
42 plt.ylabel('CV misclassification rate')
43 plt.title('10-Fold Cross-Validation: Decision Tree Depth')
44 plt.legend()
45 plt.grid(True, alpha=0.3)
46 plt.tight_layout()
47 plt.savefig('/tmp/cv_tree_depth.png', dpi=150)
48 plt.show()
49
50 print(f'Best max_depth : {best_depth}')
51 print(f'CV error at best depth: {best_cv_err:.4f}  ({(1-best_cv_err)
52     *100:.1f}% CV accuracy)')
53
54 # Fitting the tuned tree on FULL training set
55
56 tree_tuned = DecisionTreeClassifier(max_depth=best_depth, random_state
57     =1)
58 tree_tuned.fit(x_train, y_train)
59
60 y_pred_tree = tree_tuned.predict(x_test)
61 f1_tree = f1_score(y_test, y_pred_tree, pos_label='high_bike_demand')
62 train_err_tuned = np.mean(tree_tuned.predict(x_train) != y_train)
63 test_err_tuned = np.mean(y_pred_tree != y_test)
64
65 print('Tuned Decision Tree')
66 print(f'max_depth : {best_depth}')
67 print(f'Number of leaves: {tree_tuned.get_n_leaves()}')
68 print(f'Training error : {train_err_tuned:.4f}  ({(1-train_err_tuned)
69     *100:.1f}% accuracy)')
70 print(f'Test error : {test_err_tuned:.4f}  ({(1-test_err_tuned)
71     *100:.1f}% accuracy)')
72 print(f'F1 Score (Tree - High Demand): {f1_tree:.4f}')
73 print("\nClassification Report (Tree):")
74 print(classification_report(y_test, y_pred_tree))

```

```

70
71 # Confusion matrix
72 print('\nConfusion Matrix (Test Set):')
73 cm_tree = confusion_matrix(y_test, y_pred_tree, labels=['
    high_bike_demand', 'low_bike_demand'])
74 disp = ConfusionMatrixDisplay(cm_tree, display_labels=['high', 'low'])
75 fig, ax = plt.subplots(figsize=(5, 4))
76 disp.plot(ax=ax, cmap='Blues')
77 ax.set_title(f'Decision Tree (depth={best_depth}) - Confusion Matrix')
78 plt.tight_layout()
79 plt.savefig('/tmp/cm_tree.png', dpi=150)
80 plt.show()
81
82 # visualizing the Tuned Tree; For readability, we are visualising at
    most depth 4
83 viz_depth = min(best_depth, 4)
84 tree_viz = DecisionTreeClassifier(max_depth=viz_depth, random_state=1)
85 tree_viz.fit(x_train, y_train)
86
87 FEATURES = x_train.columns.tolist()
88
89 dot_data = tree.export_graphviz(
90     tree_viz,
91     out_file=None,
92     feature_names=FEATURES,
93     class_names=['high_bike_demand', 'low_bike_demand'],
94     filled=True,
95     rounded=True,
96     proportion=True
97 )
98
99 graph = graphviz.Source(dot_data)
100 graph.render('/tmp/decision_tree_viz', format='png', cleanup=True)
101 graph
102
103 # Random Forest
104
105 # Tune number of trees: when does error stabilise?
106 n_trees_range = [1, 5, 10, 25, 50, 100, 150, 200, 300]
107 rf_test_errors = []
108
109 for n in n_trees_range:
110     rf = RandomForestClassifier(n_estimators=n, random_state=1)
111     rf.fit(x_train, y_train)
112     rf_test_errors.append(np.mean(rf.predict(x_test) != y_test))
113
114 plt.figure(figsize=(9, 4))
115 plt.plot(n_trees_range, rf_test_errors, 'g-o', markersize=5)
116 plt.xlabel('Number of trees (n_estimators)')
117 plt.ylabel('Test error rate')
118 plt.title('Random Forest: Test Error vs Number of Trees')
119 plt.grid(True, alpha=0.3)
120 plt.axhline(test_err_tuned, color='blue', linestyle='--', alpha=0.6,
    label=f'Single tree (depth={best_depth})')
121 plt.legend()
122 plt.tight_layout()
123 plt.savefig('/tmp/rf_n_trees.png', dpi=150)
124 plt.show()
125 print('As n_estimators increases, error stabilises - diminishing
    returns beyond ~100 trees.')
126
127 # Tune max_features with 10-fold CV
128 # max_features controls how many features are randomly considered at
    each split
129 # sqrt(15) 4 is the standard default for classification

```

```

130
131 max_features_options = [1, 2, 3, 4, 5, 7, 10, 15] # 15 = use all
    features (= bagging)
132 n_fold = 10
133 cv = skl.ms.KFold(n_splits=n_fold, random_state=1, shuffle=True)
134
135 rf_cv_errors = np.zeros(len(max_features_options))
136
137 for train_idx, val_idx in cv.split(x_train):
138     X_tr, X_val = x_train.iloc[train_idx], x_train.iloc[val_idx]
139     y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]
140
141     for j, mf in enumerate(max_features_options):
142         rf_cv = RandomForestClassifier(n_estimators=100, max_features=
mf, random_state=1)
143         rf_cv.fit(X_tr, y_tr)
144         rf_cv_errors[j] += np.mean(rf_cv.predict(X_val) != y_val)
145
146 rf_cv_errors /= n_fold
147
148 best_mf_idx = np.argmin(rf_cv_errors)
149 best_mf = max_features_options[best_mf_idx]
150
151 plt.figure(figsize=(9, 4))
152 plt.plot(max_features_options, rf_cv_errors, 'g-o', markersize=6)
153 plt.axvline(best_mf, color='red', linestyle='--', label=f'Best
max_features = {best_mf}')
154 plt.scatter(best_mf, rf_cv_errors[best_mf_idx], color='red', s=100,
zorder=5)
155 plt.text(best_mf + 0.2, rf_cv_errors[best_mf_idx] + 0.001,
156         f'({best_mf}, {rf_cv_errors[best_mf_idx]:.3f})', color='red',
        fontsize=9)
157 plt.xlabel('max_features (features per split)')
158 plt.ylabel('CV error rate')
159 plt.title('Random Forest: 10-Fold CV      Tuning max_features')
160 plt.legend()
161 plt.grid(True, alpha=0.3)
162 plt.tight_layout()
163 plt.savefig('/tmp/rf_cv_mf.png', dpi=150)
164 plt.show()
165
166 print(f'Best max_features : {best_mf}')
167 print(f'CV error          : {rf_cv_errors[best_mf_idx]:.4f}')
168
169 # Fit final Random Forest on full training set
170 rf_final = RandomForestClassifier(
171     n_estimators=100,
172     max_features=best_mf,
173     random_state=1
174 )
175 rf_final.fit(x_train, y_train)
176
177 y_pred_rf = rf_final.predict(x_test)
178 f1_rf = f1_score(y_test, y_pred_rf, pos_label='high_bike_demand')
179 train_err_rf = np.mean(rf_final.predict(x_train) != y_train)
180 test_err_rf  = np.mean(y_pred_rf != y_test)
181
182 print('Random Forest (Tuned)')
183 print(f'n_estimators      : 100')
184 print(f'max_features      : {best_mf}')
185 print(f'Training error : {train_err_rf:.4f}  ({(1-train_err_rf)*100:.1
f})% accuracy')
186 print(f'Test error      : {test_err_rf:.4f}  ({(1-test_err_rf)*100:.1f
}% accuracy')
187 print(f'F1 Score (Random Forest - High Demand): {f1_rf:.4f}')

```

```

188 print("\nClassification Report (Random Forest):")
189 print(classification_report(y_test, y_pred_rf))
190
191 # Confusion matrix
192 cm_rf = confusion_matrix(y_test, y_pred_rf, labels=['high_bike_demand',
193             'low_bike_demand'])
194 disp = ConfusionMatrixDisplay(cm_rf, display_labels=['high', 'low'])
195 fig, ax = plt.subplots(figsize=(5, 4))
196 disp.plot(ax=ax, cmap='Greens')
197 ax.set_title('Random Forest - Confusion Matrix')
198 plt.tight_layout()
199 plt.savefig('/tmp/cm_rf.png', dpi=150)
200 plt.show()
201
202 # Feature Importance
203 importances = pd.Series(rf_final.feature_importances_, index=FEATURES)
204 importances_sorted = importances.sort_values(ascending=True)
205
206 # Colour bars by feature type
207 temporal = ['hour_of_day', 'day_of_week', 'month', 'holiday', 'weekday',
208             'summertime']
209 colors = ['steelblue' if f in temporal else 'darkorange' for f in
210           importances_sorted.index]
211
212 plt.figure(figsize=(9, 6))
213 bars = plt.barh(importances_sorted.index, importances_sorted.values,
214                 color=colors)
215 plt.xlabel('Mean Decrease in Gini Impurity')
216 plt.title('Random Forest - Feature Importance\n(Blue = temporal,
217           Orange = weather)')
218
219 # Legend
220 blue_patch = mpatches.Patch(color='steelblue', label='Temporal
221                             features')
222 orange_patch = mpatches.Patch(color='darkorange', label='Weather
223                             features')
224 plt.legend(handles=[blue_patch, orange_patch], loc='lower right')
225 plt.tight_layout()
226 plt.savefig('/tmp/feature_importance.png', dpi=150)
227 plt.show()
228
229 print('Top 5 most important features:')
230 print(importances.sort_values(ascending=False).head(5).to_string())

```

A.2.8 Model prediction using random forest

```

1 """# **Model prediction**"""
2
3 print ("Generating the Model Prediction")
4
5 # Importing the Data
6 Test_data_url = 'https://drive.google.com/uc?export=download&id=1y0-
7                 sup8TL_MrP08uw7Dj6wEilUBDrym6'
8 Test_data = pd.read_csv(Test_data_url)
9 print(f"   Shape: {Test_data.shape}")
10 print(f"   Features: {list(Test_data.columns)}")
11
12 # Making the prediction base our selected Random Forest model
13
14 print(f"\nUsing trained Random Forest model:")
15 print(f"   n_estimators: 100")
16 print(f"   max_features: {best_mf}")
17 print(f"   Model: rf_final")

```

```

18 Test_data_prediction = rf_final.predict(Test_data)
19
20 # Summary of our prediction
21 print(f"\nPrediction Summary:")
22 print(f"    Total predictions: {len(Test_data_prediction)}")
23 print(f"    Unique values: {np.unique(Test_data_prediction)}")
24 print(f"    High demand predictions: {np.sum(Test_data_prediction == '
    high_bike_demand')}")
25 print(f"    Low demand predictions: {np.sum(Test_data_prediction == '
    low_bike_demand')}")
26 print(f"    Proportion high demand: {np.mean(Test_data_prediction == '
    high_bike_demand'):.1%}")
27
28 # Converting string labels to binary format
29 # 'high_bike_demand' to 1, 'low_bike_demand' to 0
30 predictions_binary = [1 if pred == 'high_bike_demand' else 0 for pred
    in Test_data_prediction]
31
32 print(f"\nConverted to binary (1=high, 0=low):")
33 print(f"    High (1): {sum(predictions_binary)}")
34 print(f"    Low (0): {len(predictions_binary) - sum(predictions_binary)}")
35 print(f"    First 20: {predictions_binary[:20]}")
36 print(f"    Last 20: {predictions_binary[-20:]}")
37
38 np.savetxt('predictions.csv', [predictions_binary], delimiter=',', fmt
    = '%d')

```

A.2.9 Boxplot for comparison

```

1 ## -----Boxplot for cross-validation comparison of chosen models
   -----
2 n_fold = 10
3
4 x = data.drop(columns = ['increase_stock'])
5 y = data['increase_stock']
6
7 models = []
8 models.append(skl_nb.KNeighborsClassifier(n_neighbors = k))
9 models.append(LogisticRegression(solver='lbfgs', max_iter=1400))
10 models.append(skl_da.LinearDiscriminantAnalysis(solver='svd',
    shrinkage=None))
11
12 models.append(DecisionTreeClassifier(max_depth=8, random_state=1))
13 models.append(RandomForestClassifier(n_estimators=100, max_features=7,
    random_state=1))
14
15 # Initialize the error matrix - size is (10 folds, 5 models)
16 missclassification = np.zeros((n_fold, len(models)))
17
18 cv = skl_ms.KFold(n_splits = n_fold, random_state = 1, shuffle = True)
19
20 for i, (train_index, val_index) in enumerate(cv.split(x)):
21     X_train_cv, X_val_cv = x.iloc[train_index], x.iloc[val_index]
22     y_train_cv, y_val_cv = y.iloc[train_index], y.iloc[val_index]
23
24     for m in range(len(models)):
25         model = models[m]
26         model.fit(X_train_cv, y_train_cv)
27         prediction = model.predict(X_val_cv)
28         missclassification[i, m] = np.mean(prediction != y_val_cv)
29
30 # PLOT FOR 5 MODELS
31 plt.figure(figsize=(10, 6))
32 plt.boxplot(missclassification)

```

```
33 plt.xticks(np.arange(5)+1, ('kNN', 'Logreg', 'LDA', 'Decission Tree',  
    'Random Forest'))  
34 plt.ylabel('Validation error (10-fold CV)')  
35 plt.title('Model Comparison')  
36 plt.grid(axis='y', alpha=0.3)  
37 plt.show()
```